



Welcome to the ADI Interview Prep Series: Week 2

```
<p> Fall 2025. </p>
```

Overview



Week 1: Arrays + Hashing

Week 2: Two Pointers + Sliding Window

Week 3: Binary Search + Trees

Week 4: Stack + Linked List

Week 5: Heap + Priority Queue

Week 6: Graphs

Week 7: Dynamic Programming + Greedy

TODAY'S PLAN:



01

Mini lesson on two pointers & sliding window



02

Problem 1:

Best Time to Buy and Sell a Stock



03

Problem 2: container with most water



04

Problem 3: Longest Substring Without Repeating Characters



05



06



FRAMEWORK: SLIDING WINDOW

Key Components

- Two pointers represent bounds of window
- Window represented by HashSet or HashMap
- Expand window (`right++`) when conditions met
- Reduce window (`left++`) when conditions not met



FRAMEWORK: TWO POINTER

Key Components

- Two pointers move toward each other or in the same direction



Sliding Window vs Two Pointers

Two Pointers	Sliding Window
• Used when you need to optimize a brute-force approach involving two elements (e.g., pairs, subarrays) • Typically involves starting two pointers at different positions and moving them strategically • Common use cases: sorting, searching, palindrome checking, linked lists	• Used for problems that involve continuous subarrays or substrings. • Optimizes brute-force solutions by maintaining a "window" that expands and contracts dynamically. • Common use cases: maximum/minimum subarrays, longest substrings with constraints.

Key Difference:

- Two Pointers is often used on sorted arrays or lists for pair-related problems.
- Sliding Window is used when we need to track a subset (window) of data in an array or string

PROBLEM 1: BEST TIME TO BUY & SELL STOCK



You are given an integer array `prices` where `prices[i]` is the price of a given stock on the `i`th day.

You may choose a **single day** to buy stock and choose a **different day in the future** to sell it.

Return the maximum profit you can achieve.

You may choose to **not make any transactions**, in which case the profit would be 0



Example 1:

Input: `prices = [10,1,5,6,7,1]`

Output: 6

Example 2:

Input: `prices = [10,8,7,5,2]`

Output: 0

PROBLEM 1: BEST TIME TO BUY & SELL STOCK



You are given an integer array `prices` where `prices[i]` is the price of a given stock on the `i`th day.

You may choose a **single day** to buy stock and choose a **different day in the future** to sell it.

Return the maximum profit you can achieve. You may choose to **not make any transactions**, in which case the profit would be 0



****Hint 1****

We need to first buy, then sell, so you should traverse the array from left to right

****Hint 2****

If we encounter a lower price than our current buy price, what should we do?

Solution

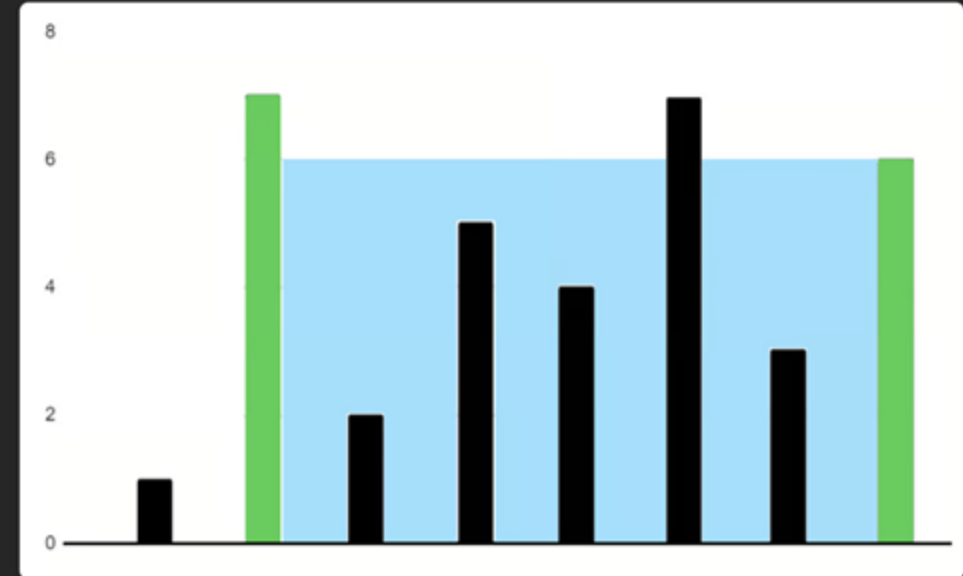
```
Java i
1 class Solution {
2     public int maxProfit(int[] prices) {
3
4         int maxP = 0;
5         int l = 0; //buy
6         int r = 1; //sell
7
8         while(r < prices.length){
9             //profitable?
10            if(prices[l] < prices[r]){
11                int profit = prices[r] - prices[l];
12                //update if greater than the current max
13                maxP = Math.max(maxP, profit);
14            } else {
15                //update left pointer if it wasn't profitable
16                l = r;
17            }
18            //update right pointer
19            r++;
20        }
21        return maxP;
22    }
23 }
24 }
25 }
```

PROBLEM 2: CONTAINER W/ MOST WATER

You are given an integer array heights where heights[i] represents the height of the ith bar.

You may choose any two bars to form a container. Return the maximum amount of water a container can store.

Example 1:



Input: height = [1,7,2,5,4,7,3,6]

Output: 36

Example 2:

Input: height = [2,2,2]

Output: 4

Solution



```
Python  i
1  class Solution:
2      def maxArea(self, heights: List[int]) -> int:
3          # two pointers: one at the left end, one at the right end
4          l, r = 0, len(heights) - 1
5          # var to store the max area found so far
6          res = 0
7
8          # keep going while the two pointers haven't met
9          while l < r:
10             # cal the area formed between the two lines:
11             # The limiting height is the shorter line,
12             # and the width is the distance between them
13             area = min(heights[l], heights[r]) * (r - l)
14             # update result if we found a larger area
15             res = max(res, area)
16
17             # move the pointer that's at the shorter line inward,
18             # since moving the taller line wouldn't help (height stays limited by the shorter line)
19             if heights[l] <= heights[r]:
20                 l += 1 # move left pointer rightward
21             else:
22                 r -= 1 # move right pointer leftward
23
24             # return the maximum area found
25             return res
26
```

PROBLEM 3: LONGEST SUBSTRING W/OUT REPEATING CHARACTERS

Given a string `s`, find the length of the longest substring without repeating characters.

Example 1:

Input: `s = "abcabcbb"`

Output: 3

Explanation: The answer is "abc", with the length of 3.

Example 2:

Input: `s = "bbbbbb"`

Output: 1

Explanation: The answer is "b", with the length of 1.

PROBLEM 3: LONGEST SUBSTRING W/OUT REPEATING CHARACTERS

Given a string `s`, find the length of the longest substring without repeating characters.

**** Hint 1:**

We can brute force this problem by checking every possible substring. However, that seems a bit too inefficient.

If we scan the string in one pass, we'd want to track the characters we've seen far. When would we need to perform updates/changes?

PROBLEM 3: LONGEST SUBSTRING W/OUT REPEATING CHARACTERS

Given a string `s`, find the length of the longest substring without repeating characters.

**** Hint 2 (efficiency):**

Say we encounter a duplicate character. How would we update our tracked characters such that only one instance remains?

Solution

```
1 class Solution(object):
2     def lengthOfLongestSubstring(self, s):
3         """
4         :type s: str
5         :rtype: int
6         """
7         l = 0
8         char_set = set()
9         max_length = 0
10        for r in range(len(s)):
11            while s[r] in char_set:
12                char_set.remove(s[l])
13                l += 1
14            char_set.add(s[r])
15            max_length = max(max_length, r-l+1)
16        return max_length
```

```
Java i
1 public class Solution {
2     public int lengthOfLongestSubstring(String s) {
3         //keep counts of how many times that character appeared
4         HashMap<Character, Integer> mp = new HashMap<>();
5         int l = 0, res = 0;
6
7
8         //loop through the string
9         for (int r = 0; r < s.length(); r++) {
10            //if the char is already there,
11            if (mp.containsKey(s.charAt(r))) {
12                //set left pointer to the max of: what's there already + 1, and the current left
13                l = Math.max(mp.get(s.charAt(r)) + 1, l);
14            }
15            //char wasn't there, so let's add it
16            mp.put(s.charAt(r), r);
17            res = Math.max(res, r - l + 1);
18        }
19        return res;
20    }
21 }
```